

CIS6003 ADVANCED PROGRAMMING

Sunrise Dental Clinic Management System - Comprehensive Technical Assessment Report

Student ID: st12345678 | Module: CIS6003 WRIT1 | Academic Year: 2025/2026 | Campus: ICBT Campus / Cardiff Metropolitan University

1. Problem Scenario & System Architectural Overview

Sunrise Dental Clinic is a high-volume private dental healthcare center located in Colombo, Sri Lanka, treating dozens of patients daily across various dental specializations including orthodontics, periodontics, pediatric dentistry, and general checkups (Pressman and Maxim, 2019). Historically, the clinic managed all operational activities—such as appointment scheduling, patient medical histories, billing calculations, and doctor consultations—using manual paper files and physical logbooks.

This traditional manual record-keeping model introduced severe operational vulnerabilities (Sommerville, 2016):

1. Double Bookings & Schedule Conflicts: Overlapping appointments for consulting dentists caused long patient waiting times and clinician friction due to lack of real-time slot visibility.
2. Lost Patient Records & Data Inconsistency: Misplaced paper files hindered historical treatment tracking, creating clinical risks and patient dissatisfaction.
3. Billing Inaccuracies & Financial Discrepancies: Manual fee calculations caused invoicing errors, misallocated consultation charges, and delayed daily revenue reconciliation.
4. Absence of Role-Based Security: Patient confidentiality and sensitive billing logs were susceptible to unauthorized access without audit logging.

To resolve these systemic inefficiencies, the management commissioned the development of the Sunrise Dental Clinic Management System. Built using Java Enterprise Edition (Java EE) web technologies, the system modernizes clinical operations through automated patient registration, auto-generated Patient IDs (PAT-0001), real-time schedule conflict prevention, automated billing calculations, interactive analytics dashboards, and role-based access control (RBAC).

2. Task A: System Design & UML Diagrams (LO I - 20 Marks)

Object-oriented system design provides the structural blueprint for developing robust, scalable software (Booch et al., 2007). The Sunrise Dental Clinic Management System utilizes Unified Modeling Language (UML 2.5) diagrams to model system requirements, domain entities, and runtime interactions.

2.1 Detailed Use Case Diagram & Actors

The system identifies three primary actors interacting with clinical boundaries:

- Clinical Staff: Registers patients, schedules appointments, verifies patient profiles, processes payments, and generates printed receipts.
- Administrator: Possesses full system privileges, managing staff accounts, configuring dentist specializations, setting treatment package costs, and reviewing financial analytics.
- System / Database: Executes background validations, auto-generates Patient IDs (PAT-0001), and enforces unique constraints.

Key Use Cases include:

- Authenticate User (<> for all secured workflows)
- Register Patient Profile (Auto-assigns PAT-0001; NIC optional for pediatric patients)
- Schedule Appointment (<> Double-Booking Validation)
- Process Payment & Generate Receipt (<> Calculate Total Cost)
- Manage System Users & Dentists (Admin Only)
- View Financial Analytics & Export CSV (Admin Only)

2.2 Class Diagram & Layered Domain Architecture

The Class Diagram models a 3-Tier Layered Architecture consisting of Model Entities, Data Access Objects (DAOs), Controllers (Servlets), and Database Connection Utilities (Gamma et al., 1994):

- Model Layer: Patient, Appointment, Bill, User, Dentist, Treatment classes with private encapsulation, getters, setters, and constructor overloads.
- DAO Layer: PatientDAO/PatientDAOImpl, AppointmentDAO/AppointmentDAOImpl, BillDAO/BillDAOImpl interfaces and implementations handling SQL persistence.
- Controller Layer: AuthServlet, DashboardServlet, PatientServlet, AppointmentServlet, BillServlet, UserServlet, DentistServlet, TreatmentServlet handling HTTP requests and session routing.
- Utility Layer: DBConnection managing JDBC connections via external db.properties configuration.

Multiplicity and Relationships:

- One Patient has 0..* Appointments (1 to Many relationship).
- One Dentist oversees 0..* Appointments (1 to Many relationship).
- One Treatment Package is linked to 0..* Appointments (1 to Many relationship).
- One Appointment generates 0..1 Bill (1 to 1 optional composition).

2.3 Sequence Diagrams

Sequence diagrams trace the dynamic chronological interactions between user interfaces, controllers, DAO layers, and the relational database (Fowler, 2004):

1. User Authentication Sequence: Staff enters credentials -> AuthServlet extracts parameters -> calls UserDAO.validateUser() -> queries MySQL via SHA-256 password hash -> returns User object -> initializes HttpSession -> redirects to /dashboard.
2. Patient Registration & Auto-ID Sequence: Staff submits patient details -> PatientServlet

validates input -> calls PatientDAO.createPatient() -> executes INSERT into patients table -> retrieves auto-increment primary key -> formats auto Patient ID (PAT-0005) -> updates DB -> displays flash success alert.

3. Appointment Booking & Conflict Validation Sequence: Staff selects dentist and date -> system queries AppointmentDAO.getBookedTimes() -> verifies selected time slot -> if conflict exists, returns error alert; if available, inserts APPT-1001 schedule.

4. Billing & Receipt Generation Sequence: Staff submits appointment number and consultation fee -> BillServlet invokes BillDAO.generateBill() -> calls MySQL Stored Procedure GenerateBill -> calculates Treatment Cost + Consultation Fee -> updates bills table with payment method and cash breakdown -> renders receipt.jsp for thermal/A4 printing.

3. Task B: System Implementation & Architectural Development (LO II - 40 Marks)

The system implementation adheres to industry-standard Java Enterprise Edition (Java EE / Jakarta EE) Web Architecture, implementing pure 100% Traditional Server-Side MVC without reliance on heavy frameworks (Spring or Hibernate) or API dependencies (Hunter, 2001).

3.1 3-Tier Model-View-Controller (MVC) Architecture

- Presentation Tier (View): Rendered using JavaServer Pages (JSP - dashboard.jsp, receipt.jsp, login.jsp) and custom Vanilla CSS styling. Employs responsive CSS Grid/Flexbox layouts, glassmorphism gradients, sticky table headers, and dedicated scrollable table containers.
- Application Tier (Controller & Business Logic): Implemented using Java HttpServlets (DashboardServlet, PatientServlet, AppointmentServlet, BillServlet, UserServlet, AuthServlet). Handles request routing, session validation, form processing, and view redirection.
- Data Access Tier (Model & Persistence): Comprises JavaBeans domain models and Data Access Objects (PatientDAOImpl, AppointmentDAOImpl, BillDAOImpl) executing parameterized SQL queries via Native JDBC.

3.2 Applied Software Design Patterns

1. Model-View-Controller (MVC) Pattern: Decouples UI representation from domain logic and HTTP request handling, promoting high cohesion and maintainability.
2. Data Access Object (DAO) Pattern: Abstracts SQL persistence operations behind clean Java interfaces, isolating database queries from Servlet controllers.
3. Singleton Pattern: Utilized within DBConnection.java to manage single-instance database connection pooling and properties initialization.
4. Front Controller / Servlet Dispatcher Pattern: Servlets intercept incoming HTTP requests, process business rules, and delegate control to JSP view templates.

3.3 Relational Database Design & Stored Procedures

The underlying MySQL relational schema consists of 5 normalized tables:

1. users (id, username, password_hash, full_name, role)
2. patients (id, patient_name, patient_id, address, phone_number, created_at)
3. dentists (id, dentist_name, specialization, contact_number)
4. treatments (id, treatment_name, cost)
5. appointments (appointment_number, patient_name, address, contact_number, dentist_name, treatment_id, appointment_date, appointment_time, status)
6. bills (bill_id, appointment_number, treatment_cost, consultation_fee, total_cost, payment_method, cash_given, balance_returned, bill_date, payment_status)

Stored Procedure Integration:

The MySQL Stored Procedure `GenerateBill` automates invoice creation:

```
```sql
CREATE PROCEDURE GenerateBill(
IN in_appointment_number VARCHAR(50),
IN in_consultation_fee DECIMAL(10, 2),
OUT out_bill_id INT,
OUT out_total_cost DECIMAL(10, 2)
)
BEGIN
DECLARE v_treatment_cost DECIMAL(10, 2);
DECLARE v_treatment_id INT;
SELECT treatment_id INTO v_treatment_id FROM appointments WHERE
appointment_number = in_appointment_number;
SELECT cost INTO v_treatment_cost FROM treatments WHERE id = v_treatment_id;
SET out_total_cost = v_treatment_cost + in_consultation_fee;
INSERT INTO bills (appointment_number, treatment_cost, consultation_fee, total_cost)
VALUES (in_appointment_number, v_treatment_cost, in_consultation_fee,
out_total_cost);
SET out_bill_id = LAST_INSERT_ID();
END;
```
```

3.4 Security & Data Protection Mechanisms

- SHA-256 Password Cryptography: All user passwords are encrypted using SHA-256 message digests prior to storage in the database.
- SQL Injection Immunity: 100% of database interactions utilize `java.sql.PreparedStatement` and `java.sql.CallableStatement` with parameterized bindings.
- Session-Based Role Access Control (RBAC): `HttpSessions` enforce role boundaries; unauthorized access redirects users to the login screen.

4. Task C: Testing & Quality Assurance Automation (LO II - 20 Marks)

System verification and quality assurance were executed using Test-Driven Development (TDD) principles and automated JUnit 4 unit testing within the Eclipse/IntelliJ IDE environments (Beck, 2003; Massol and Hatcher, 2004).

4.1 Testing Rationale & Test Automation Strategy

The test plan covers unit testing of business calculations, domain model encapsulations, DAO persistence queries, and cryptographic security routines. Automated test suites were executed using Apache Maven (`mvn test`), verifying zero regression errors.

4.2 Automated JUnit Test Suite Matrix (16 Test Cases)

The test suite consists of 16 Unit Test Cases distributed across 8 specialized Test Classes:

1. BillingCalculationTest (3 Test Cases)

- `testTotalCostCalculation`: Asserts Treatment Cost (Rs. 5000.00) + Consultation Fee (Rs. 2500.00) = Rs. 7500.00.
- `testBalanceCalculation`: Asserts Cash Given (Rs. 10000.00) - Total Cost (Rs. 7500.00) = Balance Returned (Rs. 2500.00).
- `testZeroConsultationFeeCalculation`: Verifies calculation integrity when consultation fee is Rs. 0.00.

2. PatientModelTest (2 Test Cases)

- `testPatientCreationAndGetters`: Validates encapsulation of Patient getters/setters and Patient ID code.
- `testAutoPatientIdFormat`: Verifies string formatting of auto Patient IDs (PAT-%04d => PAT-0005).

3. AppointmentModelTest (2 Test Cases)

- `testAppointmentDetailsSettersAndGetters`: Validates Appointment entity properties.
- `testAppointmentStatusChange`: Verifies state transition from 'Scheduled' to 'Completed'.

4. PatientDAOTest (2 Test Cases)

- `testGetAllPatientsNotNull`: Asserts `PatientDAO.getAllPatients()` returns non-null list.
- `testGetPatientByNicOrId`: Asserts `PatientDAO.getPatientByNic()` handles lookups cleanly.

5. AppointmentDAOTest (2 Test Cases)

- `testGetAllAppointmentsNotNull`: Verifies AppointmentDAO persistence retrieval.
- `testGetBookedTimesNotNull`: Verifies Doctor schedule conflict lookup logic.

6. BillDAOTest (2 Test Cases)

- `testGetFinancialSummaryNotNull`: Validates revenue aggregation queries.
- `testGetTreatmentRevenueReportNotNull`: Validates treatment earnings report queries.

7. SecurityPasswordHashTest (1 Test Case)

- testSHA256PasswordHashLength: Asserts SHA-256 digest produces exact 64-character hexadecimal hash strings.

8. AppTest (2 Test Cases)

- testBillingCalculationLogic: Validates basic arithmetic sanity.
- testDatabasePropertiesLoading: Asserts db.properties resource stream and DB Connection availability.

4.3 Execution Results

Execution of ``mvn test`` produced 100% Pass rate:

Tests run: 16, Failures: 0, Errors: 0, Skipped: 0 | Total time: 2.093s | BUILD SUCCESS.

5. Task D: Version Control & GitHub Repository (LO III - 20 Marks)

Version control is fundamental to modern collaborative software engineering, enabling change tracking, branch isolation, and release management (Chacon and Straub, 2014).

5.1 Repository Configuration & Remote Link

The project source code is publicly hosted on GitHub:

- GitHub Public Repository URL: <https://github.com/hiranweragoda/Sunrise-Dental.git>
- Primary Development Branches: ``main`` (Production release) and ``feature/billing-and-payments`` (Feature staging).

5.2 Version Control Workflow & Branching Strategy

Development followed a feature-branching workflow:

1. Feature Isolation: New enhancements (auto Patient IDs, unified billing tables, scrollable UI containers, JUnit tests) were developed on feature branches.
2. Commit Granularity: Changes were committed with descriptive messages reflecting atomic updates.
3. Branch Merging & Synchronization: Completed features were merged into main (``git merge feature/billing-and-payments``) and synchronized with origin (``git push origin main``).
4. Deployment Verification: Code changes were compiled into WAR packages (``mvn clean package``) and deployed to local Apache Tomcat 9 webapps.

6. References (Harvard Referencing System)

- Beck, K., 2003. Test-driven development: by example. Boston: Addison-Wesley Professional.
- Booch, G., Maksimchuk, R.A., Engle, M.W., Young, B.J., Conallen, J. and Houston, K.A., 2007. Object-oriented analysis and design with applications. 3rd ed. Boston: Addison-Wesley.
- Chacon, S. and Straub, B., 2014. Pro Git. 2nd ed. New York: Apress.
- Fowler, M., 2004. UML distilled: a brief guide to the standard object modeling language. 3rd ed. Boston: Addison-Wesley Professional.
- Gamma, E., Helm, R., Johnson, R. and Vlissides, J., 1994. Design patterns: elements of reusable object-oriented software. Reading: Addison-Wesley.
- Hunter, J., 2001. Java servlet programming. 2nd ed. Sebastopol: O'Reilly & Associates.
- Massol, V. and Hatcher, E., 2004. JUnit in action. Greenwich: Manning Publications.
- Pressman, R.S. and Maxim, B.R., 2019. Software engineering: a practitioner's approach. 9th ed. New York: McGraw-Hill.
- Sommerville, I., 2016. Software engineering. 10th ed. Harlow: Pearson Education.